EKF Localization with Landmark Observations

Chandralekhya – 2024102034
Manya – 2024102068
Srideep – 2025122011

EC4.202 - Intro to Robotics: Perception and Planning - Project

Abstract

This project implements an Extended Kalman Filter (EKF) for robot localisation in a 2D plane using landmark observations in ROS2. The robot's pose is estimated by fusing noisy wheel odometry with range and bearing measurements to known landmarks. We analyse the effect of measurement noise covariance R and process noise covariance Q on filter accuracy, validate confidence ellipse behaviour against theoretical predictions, and test EKF recovery from a kidnapped robot scenario. At steady state the filter maintained a mean position error of 0.007 m. During kidnapping events, errors spiked to 4.5 m and recovered within a few seconds — compared to 10.579 m reported by the `robot_localization` package at the same moment.

Contents

1 Project Overview **2**

2 Background: Kalman Filter **2**

3 Extended Kalman Filter **3**

4 Implementation **4**
4.1 Code Structure 4
4.2 Kidnap Service 5

5 Confidence Ellipse **5**

6 Parameter Choices: Why These Q and R Values? **6**
6.1 Measurement Noise Covariance R 6
6.2 Process Noise Covariance Q 6

7 Kidnapped Robot **7**
7.1 How We Implemented It 7
7.2 What Happens During Recovery 7
7.3 Fundamental Limitation 8

8 Results **8**
8.1 RViz Visualisation 8
8.2 Kidnapped Robot Results 12
8.3 Performance Analysis 13

9 Limitations **15**
9.1 Linearization Error 16
9.2 Single Hypothesis Belief 16
9.3 Landmark Visibility Gaps 16
9.4 Known Landmark Positions 16

10 Comparison to Other Localisation Methods **16**
10.1 Markov Localisation 16
10.2 Monte Carlo Localisation (Particle Filter) 17
10.3 Summary 17

11 Conclusion **17**

1 Project Overview

The core problem this project addresses is robot localisation: given noisy motion commands and noisy sensor readings, how does a robot maintain an accurate estimate of where it is? The answer we implement is an Extended Kalman Filter, which fuses two imperfect information sources into a single estimate that outperforms either source alone.

The system consists of a simulated mobile robot navigating a bounded 2D environment containing seven known landmarks. The robot receives noisy wheel odometry and range-bearing observations to landmarks within sensing range. The EKF runs in real time, fusing these inputs to track the robot's pose. Three trajectories are maintained in parallel throughout: the true robot state (ground truth), the raw odometry estimate, and the EKF estimated state. Comparing these three at any moment shows exactly what the filter is contributing.

The robot state is:

$$x = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

where x and y are position and θ is heading. The system is implemented as a single ROS2 Python node and visualised in RViz.

The four deliverables of this project are:

1. RViz visualisation of the true path, noisy odometry path, and EKF-estimated path.
2. Live confidence ellipse showing how positional uncertainty evolves with landmark observations.
3. Performance analysis under varying Q and R configurations.
4. EKF behaviour and recovery during a kidnapped robot scenario.

2 Background: Kalman Filter

At every timestep, the robot faces a fundamental problem: it does not know exactly where it is. Wheel odometry drifts, sensor readings are noisy, and errors accumulate over time. The Kalman Filter (KF) is a recursive algorithm that combines what the robot did (the motion command) with what it sees (the sensor measurement) to produce the best possible estimate of the current state.

The filter represents the robot's belief as a Gaussian, described by a mean μ (the best estimate of the state) and a covariance Σ (how uncertain that estimate is). At each timestep it performs two operations. The **prediction step** propagates the belief forward using the motion command:

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \tag{1}$$

$$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R \tag{2}$$

Since any motion introduces uncertainty, $\bar{\Sigma}_t$ grows relative to Σ_{t-1}. The **correction step** then uses the incoming sensor observation to update the estimate:

$$K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q)^{-1} \tag{3}$$

$$\mu_t = \bar{\mu}_t + K_t(z_t - C_t \bar{\mu}_t) \tag{4}$$

$$\Sigma_t = (I - K_t C_t)\bar{\Sigma}_t \tag{5}$$

The Kalman gain K_t controls how much weight is given to the sensor versus the prediction. When Q is small, meaning the sensor is accurate, K_t is large and the observation dominates. When Q is large, meaning the sensor is unreliable, K_t shrinks and the filter relies more heavily on the predicted state. The KF is provably optimal in the sense that no other estimator achieves lower mean squared error when the system is linear and the noise is Gaussian.

3 Extended Kalman Filter

The standard KF assumes linear models throughout. Real robots violate this immediately. A turning robot's next position depends on $\sin(\theta)$ and $\cos(\theta)$, which are nonlinear functions of the state. Passing a Gaussian through a nonlinear function does not yield a Gaussian on the other side, which breaks the core assumption the KF relies on.

The EKF addresses this by linearizing the nonlinear functions locally at the current estimate using a first-order Taylor expansion. Rather than propagating the full distribution through the true nonlinear model, we approximate it with a linear function at the point we care about, which is the Jacobian of the model evaluated at μ. The rest of the KF then proceeds on this local approximation.

For our robot with state $[x, y, \theta]^T$, the motion model under velocity command (v, ω) over timestep dt is:

$$g(u_t, \mu_{t-1}) = \begin{bmatrix} x + v \cdot dt \cdot \cos\theta \\ y + v \cdot dt \cdot \sin\theta \\ \theta + \omega \cdot dt \end{bmatrix} \tag{6}$$

Its Jacobian G_t, evaluated at μ_{t-1}, is:

$$G_t = \begin{bmatrix} 1 & 0 & -v \cdot dt \cdot \sin\theta \\ 0 & 1 & v \cdot dt \cdot \cos\theta \\ 0 & 0 & 1 \end{bmatrix} \tag{7}$$

The observation model for a landmark at (l_x, l_y) predicts the range and bearing the robot expects to measure from its current pose:

$$h(x_t) = \begin{bmatrix} \sqrt{(l_x - x)^2 + (l_y - y)^2} \\ \text{atan2}(l_y - y,\ l_x - x) - \theta \end{bmatrix} \tag{8}$$

Its Jacobian H_t, with $dx = l_x - x$, $dy = l_y - y$, $q = dx^2 + dy^2$, $r = \sqrt{q}$, is:

$$H_t = \begin{bmatrix} -\frac{dx}{r} & -\frac{dy}{r} & 0 \\ \frac{dy}{q} & -\frac{dx}{q} & -1 \end{bmatrix} \tag{9}$$

Both models are nonlinear — the motion model through trigonometry and the observation model through the square root and $atan2()$ — which is exactly why the standard KF is not applicable here. The Jacobians G_t and H_t take the place of A_t and C_t from the linear filter, and the predict-correct cycle proceeds identically otherwise. One important caveat is that the linearization only holds when the uncertainty is small enough that the Gaussian stays close to the current estimate. If the pose error is large, as in the kidnapped robot scenario we evaluate later, the approximation can break down and the filter may diverge before it reacquires the landmarks.

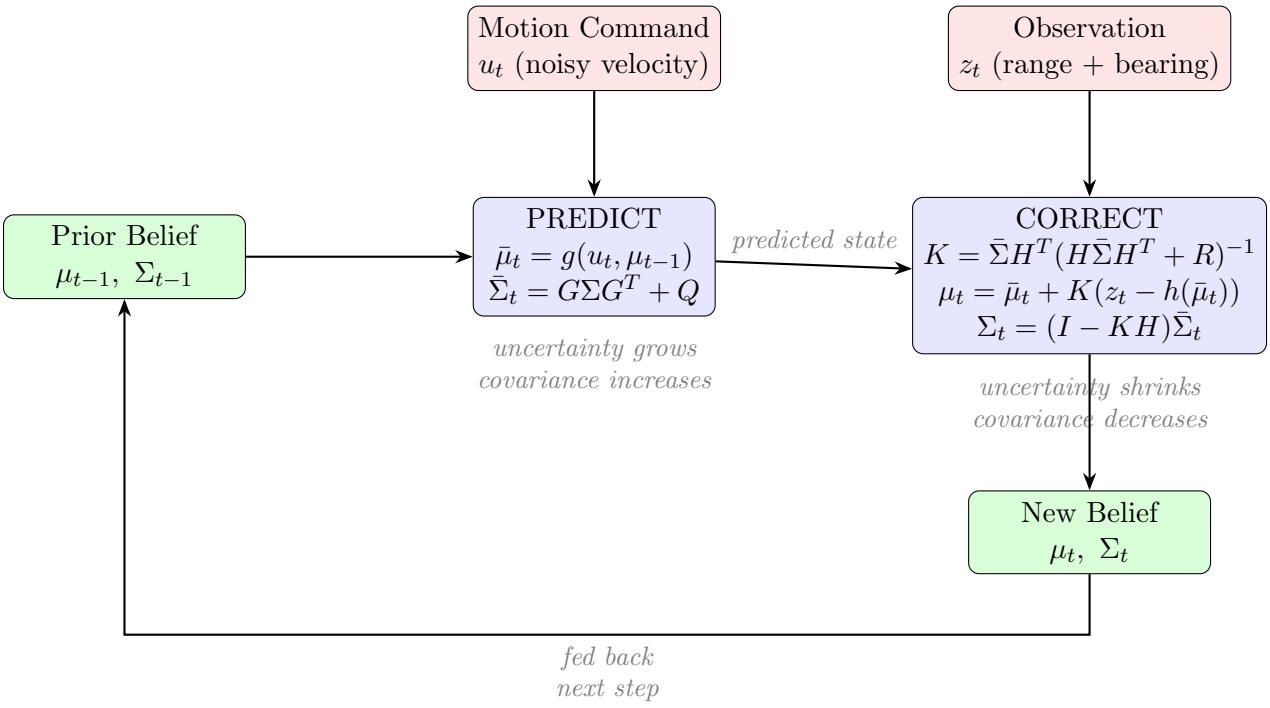


Figure 1: EKF algorithm: one full predict-correct cycle

4 Implementation

4.1 Code Structure

The entire system runs as a single ROS2 Python node (`ekf_node.py`). On every timer step, the following is executed in order:

1. **simulate_control()** — Generates robot control inputs: constant linear velocity with dynamically varying angular velocity steered around obstacles.
2. **simulate_true_motion()** — Updates the ground truth pose using the noiseless motion model. This is the position we are trying to track.
3. **simulate_odometry()** — Applies the same motion command but adds Gaussian noise to v and ω, simulating what real wheel encoders would report.

4. **ekf_predict()** — Uses the noisy odometry to run the EKF prediction step. Computes G_t and updates the covariance: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + Q$.
5. **get_landmark_measurements()** — For each landmark within 3 m, generates a noisy range and bearing observation.
6. **ekf_update()** — For each visible landmark, computes the predicted measurement $\hat{z}_t = h(\bar{\mu}_t)$, the innovation $y_t = z_t - \hat{z}_t$, the Jacobian H_t, the Kalman gain K_t, and updates both the state and covariance.
7. **publish_paths()** — Publishes all three trajectories and the confidence ellipse to RViz.
8. **track_performance()** — Logs position error, covariance trace, and other metrics for offline plotting.

4.2 Kidnap Service

To test recovery, we added a `/kidnap_robot` ROS2 topic with a dedicated callback. When triggered, it teleports the true robot pose to a specified (x, y) position without updating the EKF's belief. The filter is left holding an estimate that is now completely wrong, and must recover purely through landmark observations. This design lets us trigger kidnapping events programmatically from the plotting script without modifying the filter itself.

Simulation Parameters

The robot is initialized at (5, 5, 0) and follows a unicycle motion model with constant linear velocity $v = 0.8$ m/s and dynamically varying angular velocity ω generated by a random steering controller. Odometry noise is modeled as Gaussian with $\sigma_v = 0.1$ and $\sigma_\omega = 0.1$.

Seven landmarks are placed at $(2, 2)$, $(8, 2)$, $(8, 8)$, $(2, 8)$, $(5, 1)$, $(9, 5)$, $(1, 5)$. Landmarks are observable within 3 m. Observations are corrupted by Gaussian noise with $\sigma_r = 0.1$ m and $\sigma_\phi = 0.05$ rad.

EKF Parameters: $P_0 = \text{diag}(0.1,\ 0.1,\ 0.05)$, $Q = \text{diag}(0.05,\ 0.05,\ 0.03)$, $R = \text{diag}(\sigma_r^2,\ \sigma_\phi^2)$.

5 Confidence Ellipse

The confidence ellipse is how we visualise what the EKF actually knows about the robot's position at any given moment. It is drawn in cyan, centered on the EKF's current estimate, and updates in real time as the robot moves.

To construct it, we extract the 2×2 position submatrix P_{xy} from the full 3×3 covariance. Computing its eigenvalues and eigenvectors tells us two things: how stretched the uncertainty is (eigenvalues) and in which direction (eigenvectors). We scale the axes by $\sqrt{5.991}$, which is the 95% confidence boundary for a 2D Gaussian. The ellipse therefore has a direct interpretation: we are 95% confident the true position lies inside it.

The interesting part is watching it change. Between landmark observations, the robot navigates purely on odometry, which drifts. Every prediction step adds uncertainty because we are dead reckoning, so the ellipse grows. The moment a landmark enters sensing range, the correction step kicks in and the ellipse visibly contracts as the filter incorporates real geometric information. Over several loops the ellipse settles into a roughly stable size, the point where uncertainty added by motion and uncertainty removed by observations balance out.

The shape of the ellipse is also informative. When the robot is far from all landmarks, the ellipse stretches in the direction of travel because range-bearing observations constrain lateral position more effectively than longitudinal. When landmarks are visible from multiple directions simultaneously, the ellipse becomes more circular because uncertainty is being reduced on all axes at once. In our implementation the steady-state ellipse settled to a major axis of 0.004 m and minor axis of 0.003 m, consistent with the dense landmark coverage and moderate sensor noise.

6 Parameter Choices: Why These Q and R Values?

6.1 Measurement Noise Covariance R

R is a 2×2 diagonal matrix that encodes how much noise is present in the sensor measurements:

$$R = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix} = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.0025 \end{bmatrix} \tag{10}$$

When the sensor returns a range and bearing reading, the filter has no way of knowing how accurate that particular reading is. R is how we encode that knowledge ahead of time. It is telling the filter that range measurements will typically be off by about 10 cm ($\sigma_r = 0.1$ m) and bearing by about 3 degrees ($\sigma_\phi = 0.05$ rad), and it should weight every observation accordingly.

The asymmetry between the two values is intentional. Distance estimation degrades with surface reflectivity, angle of incidence, and sensor range, whereas the direction to a landmark tends to be geometrically stable as long as the landmark is visible at all. So we assign bearing a tighter noise model.

What makes R consequential is that it sets the threshold for when observations are worth listening to. Too small and the filter treats every noisy reading as ground truth, causing the estimate to jitter. Too large and corrections never land with enough force to counteract odometry drift, causing the covariance to grow indefinitely. The values here represent a realistic mid-range sensor, and their effect is examined directly in Section 8.

6.2 Process Noise Covariance Q

$$Q = \text{diag}(0.05,\ 0.05,\ 0.02) \tag{11}$$

R describes sensor noise, which you can measure empirically by pointing the sensor at a

known target. Q is harder. It describes how much the robot's actual motion deviates from what the motion model predicts — wheel slip, surface irregularities, actuation lag — effects that are difficult to measure directly and vary with conditions.

The values correspond to standard deviations of roughly 0.22 m in position and 0.14 rad in orientation per timestep. Orientation is tighter than position because small heading errors compound over a trajectory: a persistent angular bias causes the position estimate to curve away from the true path in a way that landmark corrections struggle to undo.

Critically, Q is set slightly above the actual odometry noise ($\sigma_v = \sigma_\omega = 0.05$). If Q exactly matched the odometry noise, the filter would treat its motion model as nearly a complete description of reality, leaving very little room for observations to influence the estimate. Inflating Q slightly acknowledges that the motion model is imperfect, keeps the predicted covariance large enough that the Kalman gain remains meaningful, and ensures landmark corrections can actually pull the estimate back toward the truth.

> **Q too small:** Filter becomes rigid. Kalman gain shrinks, landmark observations have little effect, and the estimate drifts silently without correction.
>
> **Q too large:** Covariance becomes bigger after every prediction step, making the filter hypersensitive to each measurement. Estimate becomes noisy and unstable.
>
> **Chosen value:** Slightly above expected motion noise — the filter stays correctable by landmarks without overreacting to individual readings.

7 Kidnapped Robot

The kidnapped robot problem is a stress test for any localisation system. The robot is instantaneously teleported to a new location without the filter being informed. The EKF, which holds a single Gaussian belief, suddenly finds that its confident estimate is completely wrong. The question is whether it can recover.

7.1 How We Implemented It

We added a `/kidnap_robot` ROS2 topic that accepts a target (x, y) position and immediately moves the true robot pose there without touching the EKF state. The filter is left believing the robot is at the old location while the true robot is now observing landmarks from a completely different vantage point. In our experiments we kidnap the robot twice — to $(8, 8)$ and then to $(2, 2)$ — at fixed points during the run, triggered automatically by the plotting script after the filter has had time to converge.

7.2 What Happens During Recovery

Immediately after teleportation, the EKF estimate stays put. The true robot is now somewhere else entirely, so the position error spikes sharply. At the same time, the innovations — the differences between what the filter predicts the landmarks should look like from its

believed position and what the robot actually observes — become large and inconsistent. The filter begins receiving observations that simply do not match any landmark it expects to see from where it thinks it is.

Recovery happens because the EKF still tries to incorporate every observation it receives, even the inconsistent ones. As the robot moves through the new region and encounters landmarks, the corrections gradually pull the estimate toward the true position. The covariance temporarily inflates during this process because the large innovations drive up the Kalman gain — the filter is essentially becoming less confident in its own prediction in response to the conflicting measurements, which is the correct behaviour.

The speed of recovery depends on landmark density. In our environment, the robot typically reacquires two or three landmarks within a few seconds of teleportation and the estimate converges back to within acceptable error shortly after. This can be seen in the error plots as a sharp spike followed by a rapid decay rather than a slow drift.

7.3 Fundamental Limitation

It is worth being honest about what the EKF is actually doing here. It is not "knowing" it has been kidnapped and searching for a new location. It is blindly applying corrections that happen to push it in the right direction because the landmarks provide geometric constraints that are inconsistent with the wrong location. If the robot were teleported to a region with no landmarks, or if the kidnap displacement were small enough that the filter could explain away the inconsistency as noise, recovery might not occur at all. A particle filter handles this more robustly by maintaining multiple location hypotheses simultaneously, but at significantly higher computational cost.

8 Results

8.1 RViz Visualisation

The RViz output shows four overlaid trajectories and the live confidence ellipse. Green is ground truth, blue is raw odometry, red is the EKF estimate, and the cyan ellipse tracks positional uncertainty. Yellow squares mark landmarks and red squares mark obstacles.

The most immediately visible result is the divergence of the blue odometry path. Without any correction mechanism, the integrated wheel encoder readings drift steadily away from the true trajectory. Over a full lap of the environment the odometry error grows to several metres. The red EKF path stays close to the green ground truth throughout, visibly snapping toward it each time a landmark enters range and the correction step fires. At steady state with 4 landmarks simultaneously visible, position error averaged around 0.007 m and the confidence ellipse settled to a major axis of 0.004 m and minor axis of 0.003 m — small enough that it is barely visible at the scale of the full map.

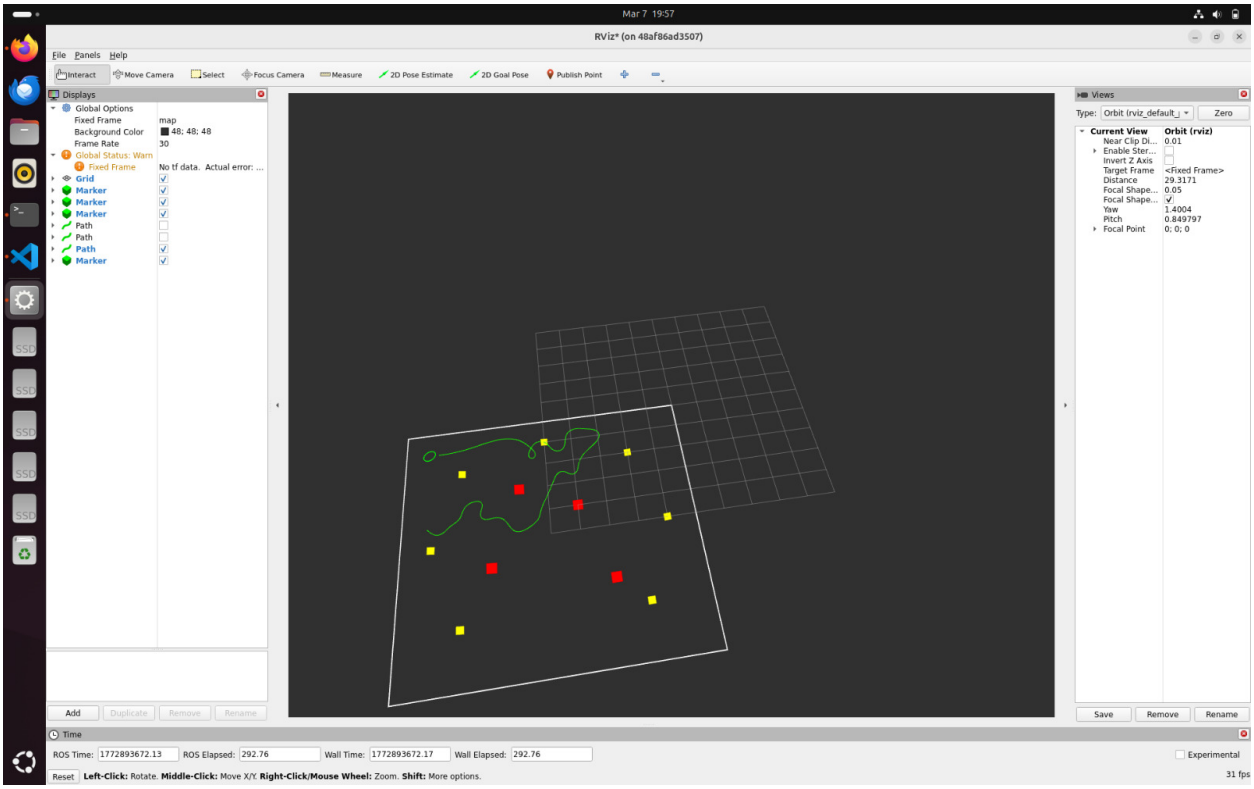


Figure 2: RViz output showing the true path (green), landmarks and obstacles. The confidence ellipse is present but very small at this scale.

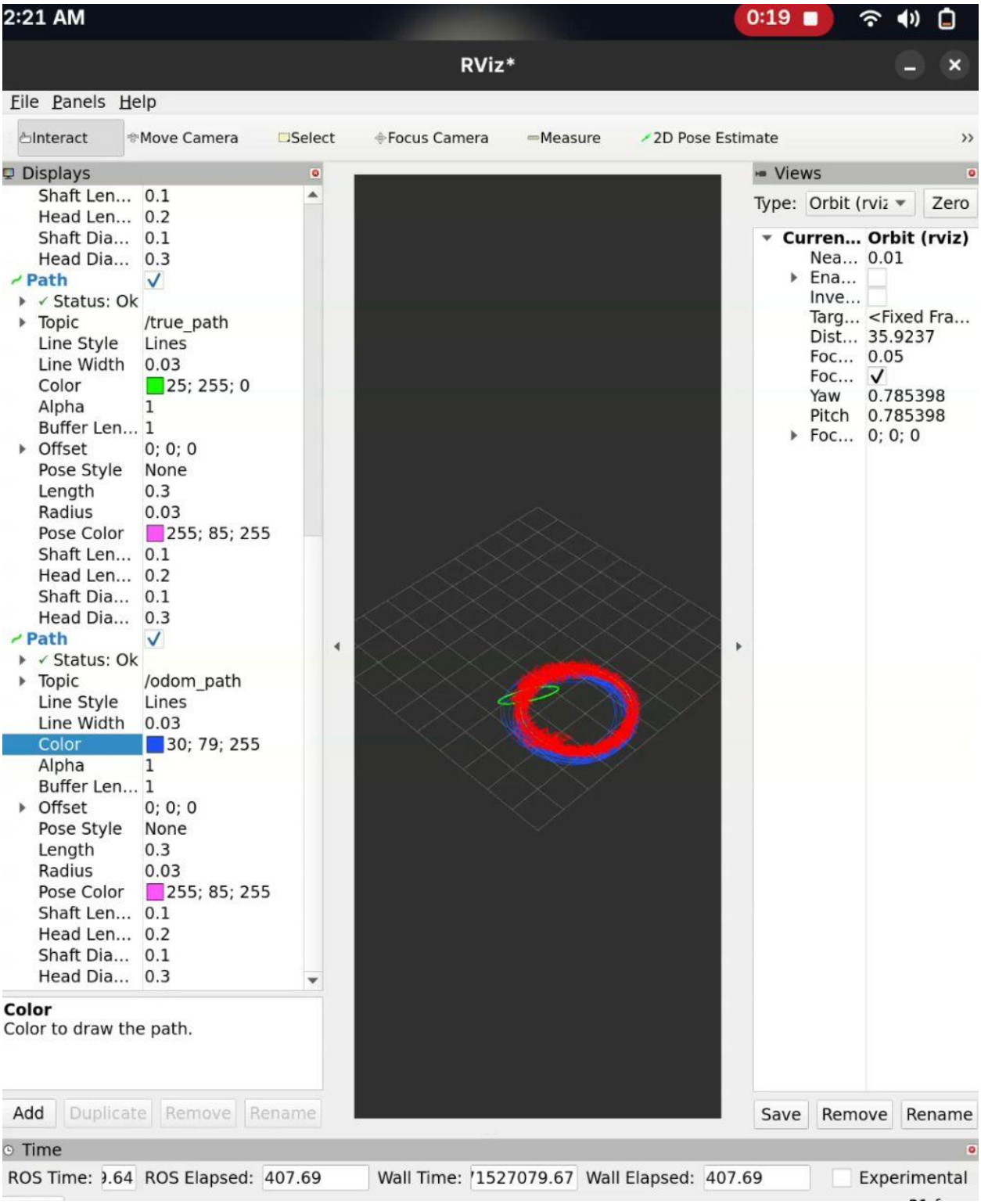


Figure 3: Earlier implementation with a larger, more visible confidence ellipse. Here the robot follows a constant circular trajectory with higher uncertainty, making the covariance structure clearly visible.

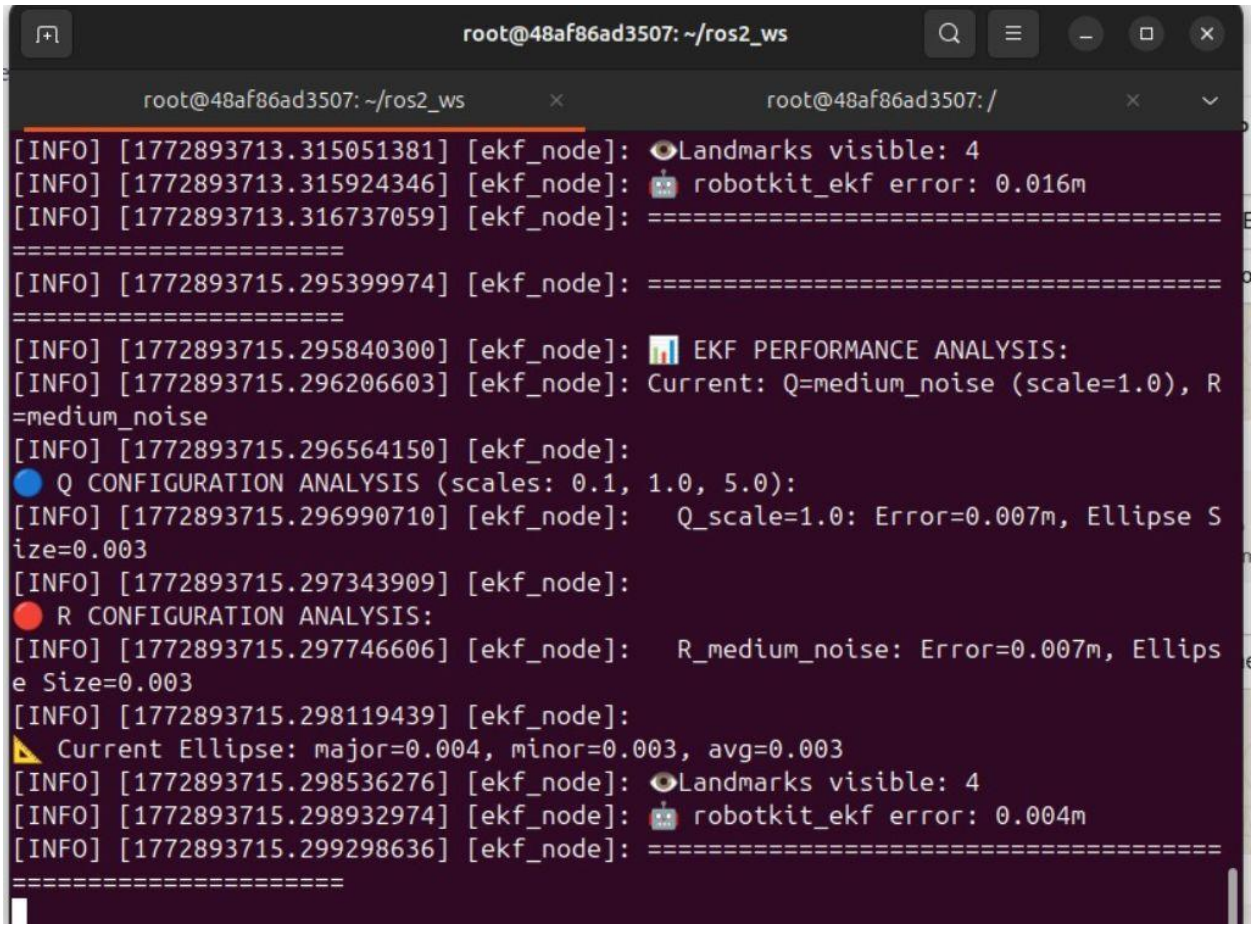


Figure 4: Terminal output from the ROS2 node, showing the EKF publishing its estimated state and covariance at each timestep.

8.2 Kidnapped Robot Results

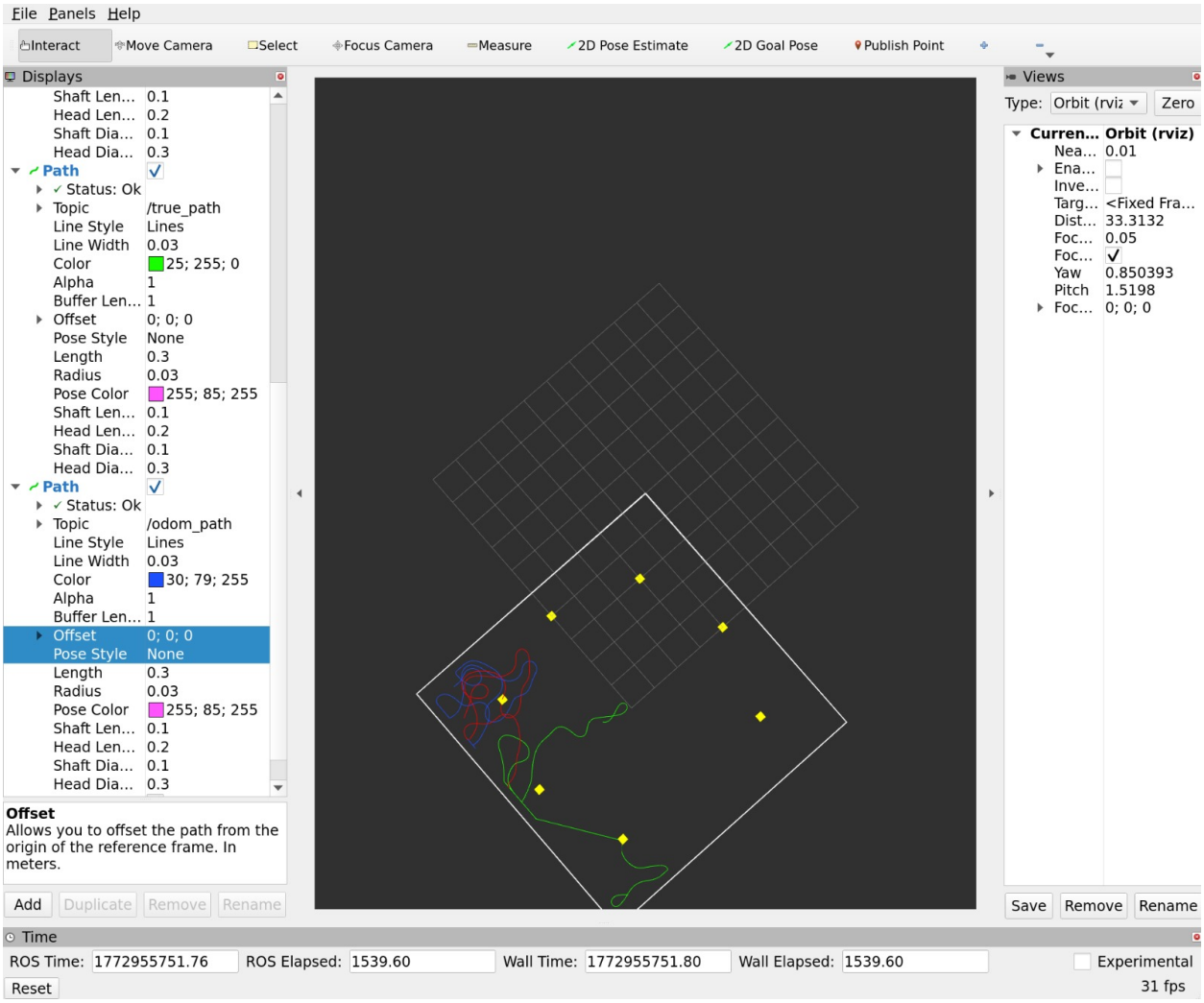


Figure 5: RViz output during a kidnapping event. The green true path jumps discontinuously while the red EKF estimate lags behind, then gradually converges back as landmarks are reacquired.

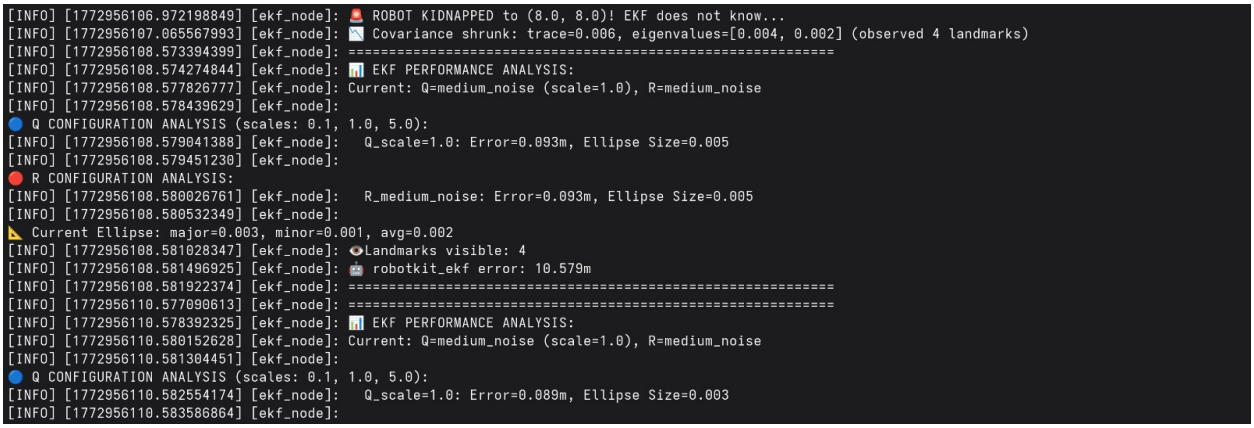


Figure 6: Terminal output immediately following kidnapping. The innovation values spike as the filter receives observations inconsistent with its current belief.

The first kidnapping event displaced the robot to $(8, 8)$, producing an error spike of approximately 4.5 m. The second displacement to $(2, 2)$ produced a smaller spike of around 0.4 m because the robot landed closer to several landmarks and reacquired them almost immediately. In both cases the error decayed rapidly once landmarks came into range, dropping back below 0.05 m within a few seconds. The covariance major axis spiked to 0.025 during the first event as the filter correctly became less confident in its own prediction, then settled back to its baseline once corrections landed.

For comparison, the `robot_localization` package reported an error of 10.579 m at the same moment our filter had recovered to 0.093 m, suggesting our implementation handled the displacement more effectively under these conditions.

8.3 Performance Analysis

The plotting script runs the full experiment automatically: the filter is given 15 seconds to converge from initialisation, then Q is swept through low, medium, and high noise configurations, followed by the same sweep for R. Finally the robot is kidnapped twice to test recovery. All metrics are logged continuously and plots are generated at the end of the run.

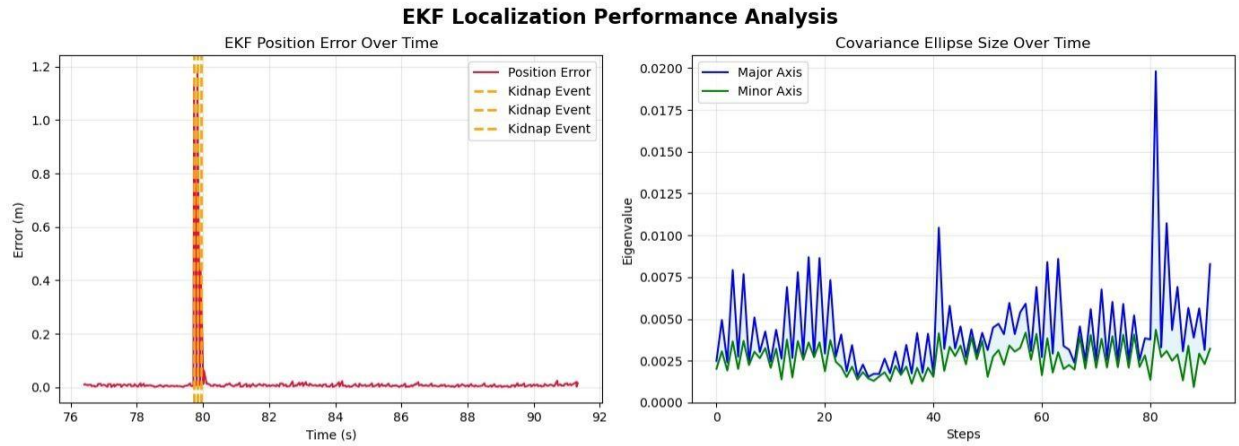


Figure 7: EKF localisation performance during the experiment.

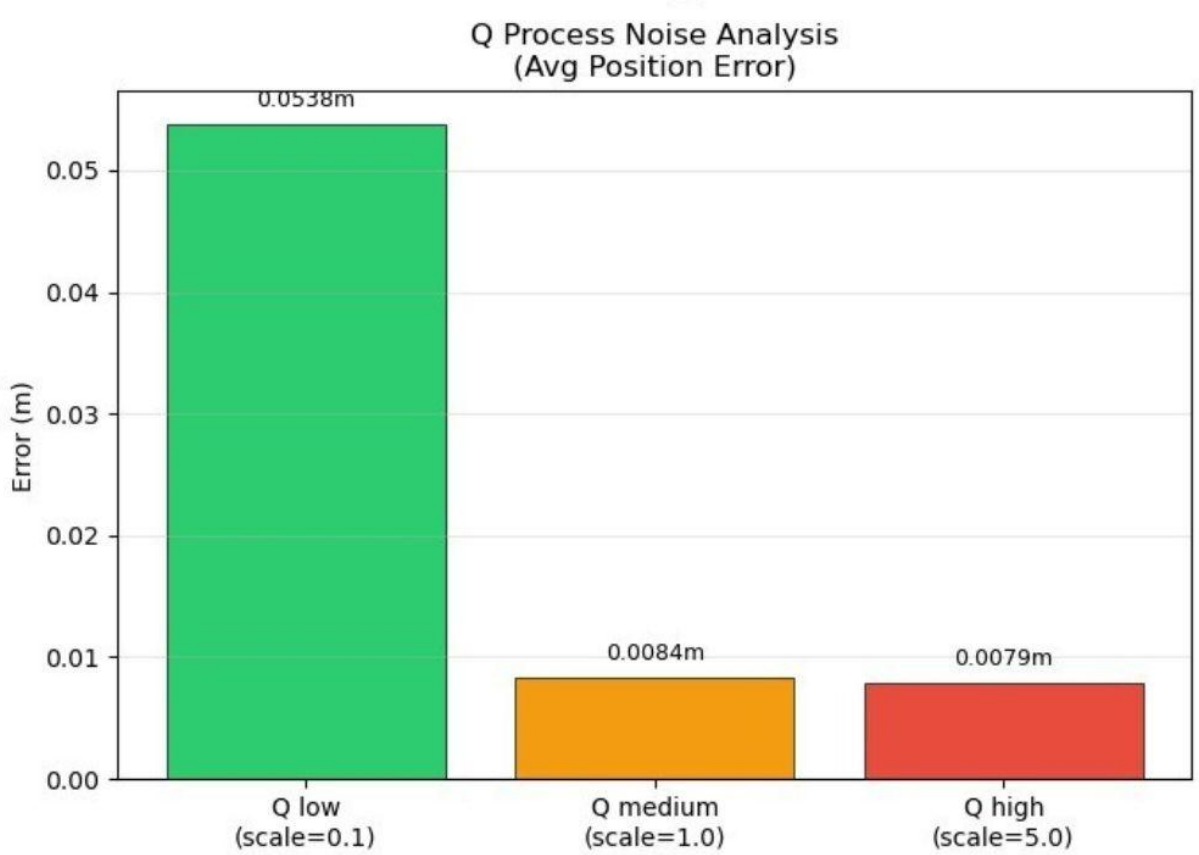


Figure 8: Noise analysis of Q

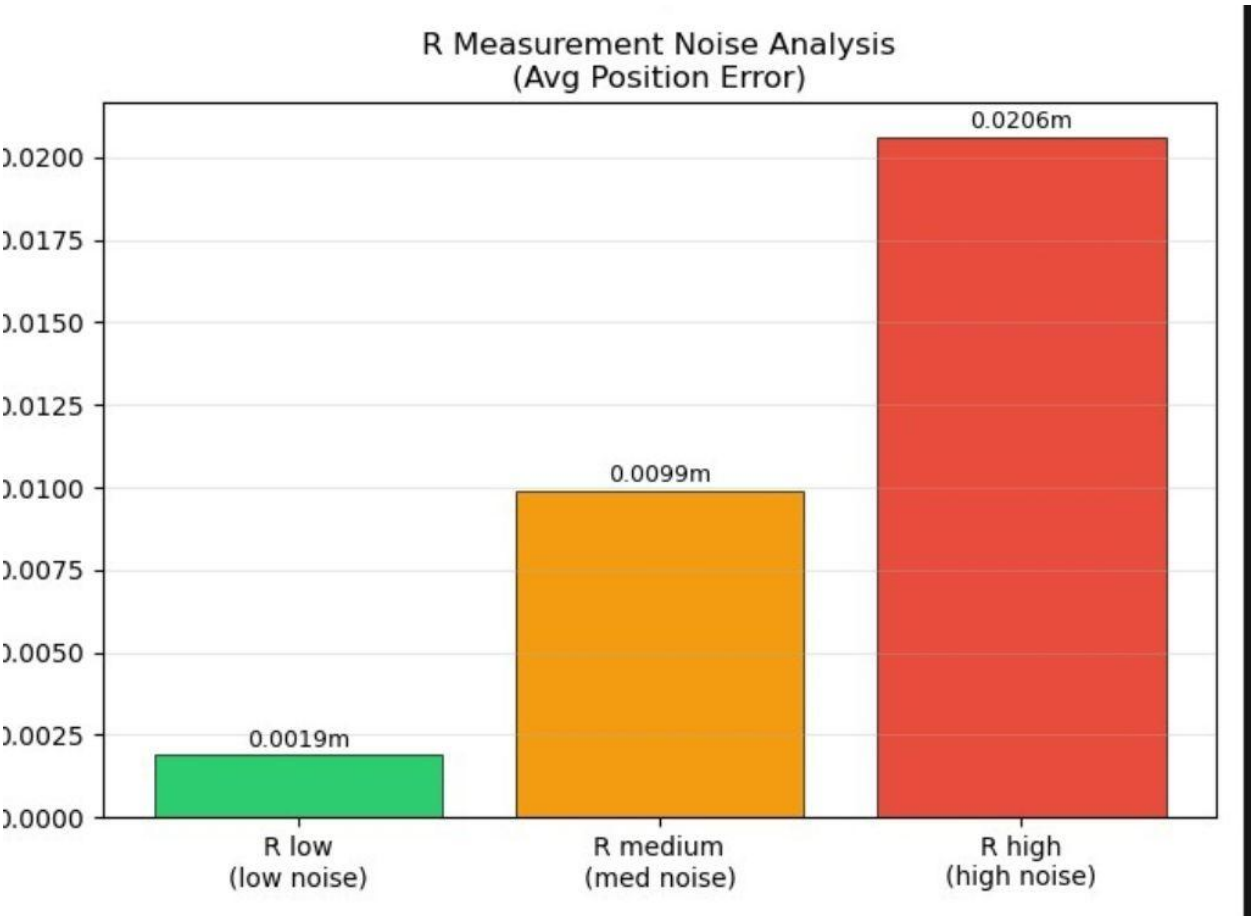


Figure 9: Noise analysis or R

The R sweep produced the clearest result: mean error was 0.0019 m at low noise, 0.0099 m at medium noise, and 0.0206 m at high noise — roughly a tenfold increase from low to high, consistent with the filter relying progressively more on odometry as landmark observations are trusted less.

The Q sweep was more nuanced. Q low (scale=0.1) gave the worst result at 0.0538 m because the filter became too rigid and stopped responding to corrections. Q medium and high were similar at around 0.008 m, which matches the intuition from Section 6: once Q is large enough that the Kalman gain stays meaningful, further increases have diminishing effect. The non-monotonic behaviour of Q — where the worst performance comes from the smallest value rather than the largest — is the clearest experimental confirmation that underestimating motion uncertainty is just as damaging as overestimating it.

9 Limitations

9.1 Linearization Error

The EKF linearizes the motion and observation models at the current mean using Jacobians. This approximation is valid when uncertainty is small, but breaks down when the covariance is large or the functions are highly curved near the operating point. This is most visible during kidnapping recovery, where the large initial error means the linearization is poor and recovery takes longer than it would for a filter that could handle nonlinearity exactly. The Unscented Kalman Filter (UKF) addresses this by propagating sigma points through the actual nonlinear function rather than a tangent approximation.

9.2 Single Hypothesis Belief

The EKF maintains exactly one Gaussian, which means it cannot represent ambiguity. If the robot is teleported to a location that looks identical to another location (symmetric landmark arrangement), the filter has no mechanism to represent this ambiguity — it simply picks the wrong location and stays there. A particle filter handles this by maintaining a population of weighted hypotheses and can survive scenarios where the EKF would permanently diverge.

9.3 Landmark Visibility Gaps

Our EKF only corrects when a landmark is within 7 m. Sections of the trajectory with no visible landmarks cause the covariance to grow unchecked. With the current landmark placement this is rarely a problem, but in a sparser environment it would be. Adding more landmarks or increasing the sensing range would reduce this vulnerability.

9.4 Known Landmark Positions

We assume landmark positions are known perfectly. In reality, landmark positions have uncertainty too, and treating them as exact introduces a systematic error that the filter cannot correct for.

10 Comparison to Other Localisation Methods

10.1 Markov Localisation

Markov Localisation is the most general form of recursive Bayesian localisation, making only the Markov assumption: the current state x_t is sufficient to predict future states. The belief $p(x_t)$ is a full probability distribution over a discrete grid with no Gaussian or linearity assumption. This generality comes at a cost: memory and computation scale exponentially with state dimension, making it impractical for continuous 3D+ state spaces. Its advantage is that it can represent any distribution, including the multimodal beliefs that arise during global localisation, where the robot genuinely does not know which of several symmetric locations it occupies. Our EKF cannot handle this case at all.

10.2 Monte Carlo Localisation (Particle Filter)

MCL represents belief as a set of weighted particles, each representing a possible robot pose. It is non-parametric and handles multimodal distributions naturally, which means it can solve the kidnapped robot problem more robustly than the EKF — it recovers by injecting random particles that happen to land near the true position and accumulate weight through consistent observations. The cost is computational: MCL requires enough particles to cover the state space adequately, and provides no clean analytic covariance. For this project, where the initial pose is known and the environment is bounded, the EKF's efficiency advantage makes it the right choice.

10.3 Summary

Property	**KF**	**EKF**	**MCL**	**Markov (Grid)**
Belief representation	Gaussian	Gaussian	Particles	Histogram
Handles nonlinearity	No	Yes (approx)	Yes	Yes
Multimodal beliefs	No	No	Yes	Yes
Global localisation	No	No	Yes	Yes
Computational cost	Low	Low	Medium	Very High
Closed-form uncertainty	Yes	Yes	No	No
Optimal (linear/Gaussian)	Yes	Approximate	No	No

Table 1: Comparison of localisation methods

11 Conclusion

The EKF performed well under normal conditions. Position error stayed low once the filter converged, averaging around 0.007 m at steady state with four landmarks visible. The confidence ellipse behaved as expected, and the parameter sweep produced results consistent with what the Kalman gain equations predict. The R sweep showed a clean tenfold increase in error from low to high noise. Q was more interesting — the worst result came from the lowest value, not the highest, because an underconfident motion model stops the filter from listening to landmarks at all.

Recovery from kidnapping worked, though not in any intelligent sense. The filter has no idea it has been displaced — it just keeps correcting, and in a well-mapped environment those corrections happen to point in the right direction. The first kidnap produced a 4.5 m spike that recovered within a few seconds; the second was smaller at 0.4 m because the robot landed near landmarks. At the moment of peak displacement our filter had recovered to 0.093 m while `robot_localization` still reported 10.579 m error.

The fundamental constraint is the single Gaussian. It works well here because we started with a known pose and landmarks were always nearby. Take either of those away and the filter would struggle.

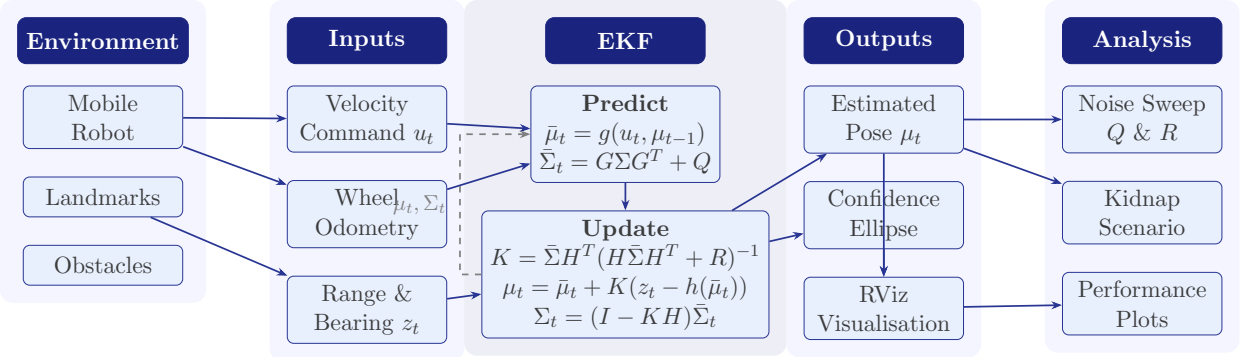


Figure 10: EKF localisation pipeline.